

NOTE: Docente: Prof. Matias Gel

NOTE: Dirigida a: Alumnos del IF009 Semanas 23

NOTE: TP asociado: TP2 Python + Autograding

Objetivos de esta guía

Al finalizar este módulo, el estudiante deberá poder:

#	Objetivo	Como se evalúa
OA1	Describir el modelo ágil y sus herramientas de desarrollo	Flujo de trabajo del TP2: commits explícitos y CI en verde
OA2	Implementar funciones en Python 3.13 con tipos, condicionales y bucles	Scripts 'src/' del TP2 con pruebas automatizadas en verde
OA3	Aplicar colecciones para resolver problemas concretos y mantener claridad estructural	Soluciones del TP2 con colecciones apropiadas y legibles
OA4	Organizar código en módulos con docstrings y type hints	Estructura 'src/tests/' del TP2 y documentación de funciones
OA5	Justificar el diseño del código y explicar su comportamiento	Comentarios técnicos y documentación de la solución

Conceptos previos necesarios

- [x] Cuenta GitHub activa con SSH configurado (Tema 00)
 - [x] GitHub Codespaces funcional (Tema 00)
 - [x] Git básico: add, commit, push (Tema 00)
 - [x] Bootstrap 5 + HTML básico (Tema 00 solo de contexto)
-

PARTE 1: El Modelo Ágil

1.1 ¿Qué es el desarrollo ágil?

El desarrollo ágil es un enfoque de ingeniería de software que prioriza iteraciones breves, retroalimentación continua y colaboración constante. Se fundamenta en la entrega temprana de valor y en la adaptación progresiva del producto.

El Manifiesto Ágil de 2001 resume esta filosofía en cuatro valores básicos:

Valoramos más	Que
Individuos e interacciones	Procesos y herramientas
Software funcionando	Documentación exhaustiva
Colaboración con el cliente	Negociación de contratos
Respuesta al cambio	Seguir un plan rígido

NOTE: Nota: Agil no implica ausencia de planificacion; implica planificacion iterativa y ajuste continuo.

1.2 Cascada vs. Agil

El modelo cascada propone fases secuenciales: requisitos, disenyo, implementacion, prueba y despliegue. Su principal limitacion es que el cliente no observa el resultado hasta etapas avanzadas del proceso.

En contraste, el enfoque agil busca validar incrementos de software de forma temprana mediante iteraciones cortas. En el TP2 esto se expresa mediante ciclos de trabajo que combinan desarrollo, prueba y ajuste continuo.

1.3 El entorno de desarrollo agil

Para este curso se utiliza GitHub Codespaces como entorno estandar. Esto garantiza que todos los estudiantes trabajen con la misma configuracion y minimiza diferencias de entorno.

El entorno se define en .devcontainer/devcontainer.json
El proyecto se ejecuta en un contenedor reproducible

Extensiones VS Code relevantes

Extension	Proposito
Pylance	Comprobacion estatica de tipos en tiempo real
Ruff	Lintor automatico y correccion de estilo
GitLens	Visualizacion del historial Git en el editor

PARTE 2: Python 3.13 Fundamentos

2.1 Por que Python 3.13

Python 3.13 introduce mejoras que facilitan el aprendizaje y reducen el esfuerzo de depuracion. Entre las novedades relevantes se encuentran errores mas claros y mayor consistencia en el comportamiento del lenguaje.

Para iniciar el interprete en Codespaces:

```
python3
```

2.2 Tipos de datos primitivos

Python es un lenguaje de tipado dinamico, pero en este curso se exige el uso de anotaciones de tipo para mejorar la claridad y la mantenibilidad.

Tipo	Ejemplos	Caracteristicas
'int'	'42', '-7', '1000000'	Precision arbitraria
'float'	'3.14', '-0.5', '1e10'	Representacion IEEE 754
'str'	'"hola"', 'mundo'	Inmutable
'bool'	'True', 'False'	Subclase de 'int'

'None' | 'None' | Indica ausencia de valor

```
nombre: str = "Ada Lovelace"
edad: int = 25
promedio: float = 7.85
activo: bool = True
email: str | None = None
```

2.3 Operadores aritmeticos

```
10 + 3    # 13
10 - 3    # 7
10 * 3    # 30
10 / 3    # 3.3333... # division real
10 // 3   # 3 # division entera
10 % 3    # 1 # resto
2 ** 8    # 256 # potencia
```

NOTE: Precaucion: 10 / 2 produce 5.0, no 5. Use // si se requiere un entero.

2.4 Strings inmutabilidad e interpolacion

Los strings en Python son inmutables. Para obtener una variante distinta se debe crear un nuevo objeto.

```
saludo = "hola"
saludo_formal = saludo.capitalize()
print(saludo) # "hola"
```

f-strings

```
nombre = "Ada"
edad = 25
texto = f"Hola, {nombre}. Tiene {edad} anos."
```

2.5 Variables y referencias

En Python, las variables son etiquetas que apuntan a objetos en memoria.

```
a = [1, 2, 3]
b = a
b.append(99)
print(a) # [1, 2, 3, 99]
```

Para evitar modificaciones compartidas, copie explícitamente el objeto:

```
c = a.copy()
c.append(100)
print(a) # [1, 2, 3, 99]
```

2.6 None y bool

```
resultado = None
if resultado is None:
    print("Sin resultado aun")

if not []:
    print("Lista vacia")
```

PARTE 3: Control de flujo

3.1 Condicionales if / elif / else

```
def clasificar_nota(nota: int) -> str:
    if nota >= 90:
        return "Sobresaliente"
    elif nota >= 70:
        return "Aprobado"
    elif nota >= 50:
        return "Regular"
    else:
        return "Desaprobado"
```

3.2 Bucles for

```
frutas = ["manzana", "banana", "pera"]
for fruta in frutas:
    print(fruta.upper())
```

Si se requiere el indice, utilice enumerate:

```
for i, fruta in enumerate(frutas, start=1):
    print(f"{i}: {fruta}")
```

3.3 Bucles while

```
intentos = 0
MAX = 3

while intentos < MAX:
    clave = input("Clave: ")
    if clave == "1234":
        print("Correcto")
        break
    intentos += 1
else:
```

```
print("Bloqueado por demasiados intentos")
```

3.4 match Pattern Matching

```
def describir_http(codigo: int) -> str:
    match codigo:
        case 200:
            return "OK exito"
        case 400:
            return "Bad Request error del cliente"
        case 404:
            return "Not Found recurso inexistente"
        case 500:
            return "Internal Server Error"
        case _:
            return f"Codigo HTTP: {codigo}"
```

PARTE 4: Funciones

4.1 Anatomia completa de una funcion

```
def calcular_imc(peso_kg: float, altura_m: float) -> float:
    """
    Calcula el Indice de Masa Corporal.

    Args:
        peso_kg: Peso en kilogramos (debe ser mayor que 0).
        altura_m: Altura en metros (debe ser mayor que 0).

    Returns:
        IMC como numero flotante.

    Raises:
        ValueError: Si algun parametro es menor o igual a 0.
    """
    if peso_kg <= 0 or altura_m <= 0:
        raise ValueError("Peso y altura deben ser positivos")
    return peso_kg / (altura_m ** 2)
```

4.2 Parametros

```
def crear_perfil(
    nombre: str,
    edad: int,
    rol: str = "alumno"
) -> dict[str, str | int]:
    return {"nombre": nombre, "edad": edad, "rol": rol}
```

4.3 Valores de retorno

```
def cuadrado(n: int) -> int:
    return n ** 2

def dividir(a: int, b: int) -> tuple[int, int]:
    return a // b, a % b

def buscar(lista: list[int], valor: int) -> int | None:
    for i, x in enumerate(lista):
        if x == valor:
            return i
    return None
```

PARTE 5: Colecciones

5.1 list secuencia mutable

```
notas = [85, 72, 91, 68, 77]
notas.append(95)
notas.insert(0, 100)
ultimo = notas.pop()
```

5.2 tuple secuencia inmutable

```
coordenada = (3.5, -1.2)
x, y = coordenada
```

5.3 dict mapa clave-valor

```
alumno = {
    "nombre": "Ana",
    "legajo": 12345,
    "notas": [85, 90]
}
```

5.4 set conjunto sin duplicados

```
lenguajes = {"Python", "JavaScript", "Python"}
```

5.5 Comprensiones

```
cuadrados = [x ** 2 for x in range(10)]
pares = [x for x in range(20) if x % 2 == 0]
```

5.6 Funciones de iteracion

```
for i, fruta in enumerate(["manzana", "banana", "cereza"], start=1):
    print(f"{i}: {fruta}")

for nombre, nota in zip(["Ana", "Bob", "Carlos"], [85, 72, 91]):
    print(f"{nombre}: {nota}")
```

PARTE 6: Modulos, Funciones de Orden Superior y Decoradores

6.1 Modulos y paquetes

```
import calculadora
resultado = calculadora.sumar(5, 3)
```

6.2 Funciones de orden superior

```
def aplicar(func, lista: list[int]) -> list[int]:
    return [func(x) for x in lista]
```

6.3 Lambdas

```
cuadrado = lambda x: x ** 2
```

6.4 Decoradores basicos

```
import functools

def registrar(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        resultado = func(*args, **kwargs)
        return resultado
    return wrapper
```

PARTE 7: Type Hints (Python 3.10+)

7.1 Que son y por que importan

Las anotaciones de tipo documentan las expectativas de entrada y salida de funciones. No se validan en tiempo de ejecucion, pero contribuyen a la calidad del codigo y a la

deteccion temprana de errores.

7.2 Sintaxis moderna

```
notas: list[int]
mapa: dict[str, float]
email: str | None = None
```

7.3 Anotaciones en funciones

```
def procesar_alumnos(
    nombres: list[str],
    notas: dict[str, int],
    umbral: int = 70
) -> tuple[list[str], list[str]]:
    aprobados = [n for n in nombres if notas.get(n, 0) >= umbral]
    desaprobados = [n for n in nombres if notas.get(n, 0) < umbral]
    return aprobados, desaprobados
```

7.4 Callable

```
from collections.abc import Callable

def aplicar_a_todos(
    lista: list[int],
    func: Callable[[int], int]
) -> list[int]:
    return [func(x) for x in lista]
```

PARTE 8: PEP 8 + Ruff Estilo de Código

8.1 Las reglas mas importantes

Regla	Mal	Bien
Identacion	2 espacios o tabs	4 espacios
Nombres de funciones	'MiFunc'	'mi_func'
Nombres de clases	'mi_clase'	'MiClase'
Constantes	'maxIntentos'	'MAX_INTENTOS'
Largo de linea	>88 caracteres	maximo 88 caracteres

8.2 Ruff el linter del curso

```
ruff check .
ruff check --fix .
ruff check src/calculadora.py
```

Se recomienda ejecutar Ruff antes de cada entrega para identificar violaciones de estilo y

errores sintacticos simples.